Mikhail Razakov

Compiler & Program Analysis Engineer

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

Compiler and program-analysis engineer with professional experience at MCST and ISP RAS. At MCST, I implemented LLVM 22 LICM and an interprocedural global-IV prototype with a separate legality phase; I now contribute to SharpChecker. Public projects demonstrate LLVM IR transformations with a 29-case regression suite, an x86-64 backend with register allocation, and Roslyn fixed-point data flow.

EXPERIENCE

2026 — present — **ISP RAS — Static Analysis Engineer**

- Contribute to SharpChecker, ISP RAS's industrial static-analysis platform for C#/.NET.

July — August 2026 — **MCST — Compiler Engineering Intern**

- Implemented an LLVM LICM pass with loop analysis, side-effect/speculative-safety checks, and hoisting of loop-invariant instructions to preheaders.
- Developed a conservative interprocedural global-IV prototype with APInt affine evolution, transitive call effects, a separate legality phase, and LLVM IR transformation; unsupported CFG/call cases are rejected.

PROJECTS

LLVM Interprocedural Global IV Optimization

An LLVM 22 pass localizes supported induction-like globals after interprocedural effects/affine analysis and a separate legality decision; APInt models fixed-width wraparound.

29 positive/negative regression cases: global-iv,verify, structural expectations, idempotence, and observable before/after execution comparison.

x86-64 Codegen & Register Allocation Lab

A compiler backend with SSA/CFG validation, liveness/interference analysis, linear-scan and seeded simulated-annealing allocators, spills, phi lowering, and a SysV x86-64 emitter.

An independent assignment verifier plus native-vs-interpreter differential execution; tests cover branches, loops, spills, and parallel phi moves.

DerefAfterNullAnalyzer

A Roslyn ControlFlowGraph analyzer propagates null state over conditional edges to a fixed point, handles loops/back edges, and merges states from multiple predecessors.

Handles method/property/index/event dereferences and invalidation after assignment/ref/out; dedicated analyzer tests use Microsoft.CodeAnalysis.CSharp.Testing.

UniversalToolchain

Modular compiler/runtime infrastructure: typed artifact contracts, dependencies/conflicts, provider selection, pass ordering, and artifact routes compile into an immutable LanguagePlan before execution.

Published UniversalToolchain.Wist alpha package; interpreter/CIL paths, exact package binding, and fail-closed runtime checks protect the selected execution plan.

SKILLS

Compilers
C++23, LLVM 22, LLVM IR, optimization passes, APInt, interprocedural analysis, CFG/SSA, dominators, loop analysis, LICM, legality, LLVM Verifier

Program Analysis
C#, .NET, Roslyn ControlFlowGraph, fixed-point data flow, conditional edges, loops/back edges, state propagation and joins

Backend / Codegen
Rust, SysV x86-64, liveness/interference, linear scan, simulated annealing, spills, phi lowering, differential execution

Compiler Infrastructure
typed artifact contracts, deterministic planning/pass ordering, artifact routing, interpreter/CIL parity, backend/runtime composition

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

LangDev'26 — Build the Language, Then Make the Abstractions Disappear
Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.

UniversalToolchain — Baltic Science and Engineering Competition
First Degree Diploma and Grand Prize "Perfection as Hope" for UniversalToolchain.

MEPhI Junior
Overall winner; First Degree Diploma, 96/100 for UniversalToolchain.

Moscow / remote